ADR-0007 — Quelle source pour les données de l'ERP

Status : Accepté
Date : 29/10/2026

Contexte

Le modèle doit être entraîné sur les commandes de l'ERP plutôt que sur des données synthétiques. L'ERP (simulateur fourni) écrit les mêmes commandes, avec les mêmes `order_id`, dans deux bases :

- **PostgreSQL** : tout l'historique depuis le 1er janvier 2015, puis le flux courant.
- **MongoDB** : l'historique depuis 2023 seulement, puis le même flux courant.

L'historique garde la trace des migrations de schéma, `weekend` vaut `NULL` avant juillet 2015, `preparation_time_min` avant juillet 2017. Aucune des deux bases ne contient la cible `express_eligible`, l'ERP décrit les commandes, pas leur mode de livraison. Ses valeurs ne suivent pas non plus le contrat `openapi.yml`, figé (`rain` au lieu de `pluie`, `carrier_capacity` de 5 à 50 au lieu de 0 à 1…). Il faut choisir une seule base comme source.

Options considérées

Option	Avantage	Inconvénient
PostgreSQL	Historique complet (2015 →), donc plus de données d'entraînement ; schéma fixe, identique au DataFrame du modèle ; même moteur que le catalogue DuckLake (ADR-0001), déjà en production sur le VPS	Requêtes SQL à écrire ; colonnes `NULL` sur l'ancien historique à traiter
MongoDB	Documents souples, aucune migration à gérer ; upsert simple	Historique limité à 2023 → ; mêmes commandes que PostgreSQL, sans cible non plus ; schéma non garanti d'un document à l'autre ; second moteur à héberger, sauvegarder et sécuriser

Décision

Les données sont intégrées depuis **PostgreSQL**, en lecture seule, par une implémentation de `OrdersSource` (`PostgresOrdersSource`) qui lit la table `orders` et la traduit vers les données du modèle :

- `weather` : `clear` → `normal`, `rain` → `pluie`, `snow` → `neige`, `wind` → `orage` ;
- `delivery_zone` : `suburb` → `banlieue`, `rural` → `rurale` ;
- `customer_type` : `individual` → `standard`, `business` → `premium` ;

- `carrier_capacity` divisée par 50, le maximum produit par l'ERP ;
- `weekend` manquant déduit de `day_of_week` ;
- cible `express_eligible` calculée par la règle métier du jeu synthétique.

Conséquences

- Le modèle s'entraîne sur près de dix ans de commandes au lieu de trois.
- **La cible est simulée** : le modèle réapprend la règle métier, ses métriques ne mesurent pas une performance réelle. Une vraie cible demande une donnée de livraison (délai réel, mode de livraison) absente de l'ERP.
- Les commandes antérieures à juillet 2017 sont écartées faute de `preparation_time_min` : environ 7 400 commandes sur 11 000 sont gardées.
- Deux traductions sont des hypothèses à faire valider par le métier, le vent compté comme un orage, et 50 comme capacité maximale du transporteur (une valeur au-delà donnerait plus de 1 et la commande serait écartée par le nettoyage).
- Les identifiants de la base passent par des variables d'environnement (`ERP_POSTGRES_*`), jamais en dur, sans `ERP_POSTGRES_HOST`, l'entraînement garde les données synthétiques.
- Un seul moteur de base de données à exploiter sur le VPS, la base de l'ERP et le catalogue DuckLake sont deux bases distinctes sur le même serveur PostgreSQL, l'application lit l'ERP avec un rôle en lecture seule.
- DuckLake reste le stockage des commandes reçues par l'API (ADR-0001), l'ERP est une source pour l'entraînement. L'API ne lit jamais la base de l'ERP.
- MongoDB n'est pas mis en service : ni sur le VPS, ni dans les environnements de test et de production. Seul PostgreSQL est déployé.